

PEMROGRAMAN BERORIENTASI OBJEK

Laporan Ujian Tengah Semester

KantinPay ITK

https://github.com/augustagozzal/KANTIN-PAY_UTS-PBO.git



Dosen Pengampu :

Himawan Wicaksono, S.ST., M.T.

Disusun Oleh :

RIZKY AUGUSTA GOZZAL

04231078

**PROGRAM STUDI TEKNIK ELEKTRO
JURUSAN TEKNIK ELEKTRO, INFORMATIKA, DAN BISNIS
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN
2025**

TAHAP 1: ANALISIS SDLC & MANAJEMEN PROYEK

1. Jelaskan secara ringkas, mengapa sistem pada studi kasus Anda ini lebih aman dan cocok menggunakan paradigma Berorientasi Objek (OOP) dibandingkan paradigma Terstruktur (Prosedural)! Berikan contoh masalah fatal yang mungkin terjadi jika data sensitif pada kasus Anda dibiarkan sebagai variabel global.

Jawab :

Sistem KantinPay ITK dirancang menggunakan paradigma Pemrograman Berorientasi Objek (OOP) karena menawarkan tingkat keamanan dan perlindungan data yang jauh lebih tinggi dibandingkan dengan paradigma Terstruktur atau Prosedural. Keunggulan utama OOP dalam kasus ini terletak pada konsep Enkapsulasi, di mana data sensitif seperti Saldo Mahasiswa dan PIN tidak dibiarkan terbuka, melainkan dibungkus di dalam sebuah *class* dan dilindungi menggunakan modifier *private*. Hal ini memastikan bahwa data tersebut hanya dapat diakses atau diubah melalui metode resmi yang telah didefinisikan secara ketat dalam sistem.

Sebagai perbandingan, jika sistem ini dibangun dengan paradigma Terstruktur, data sensitif kemungkinan besar akan diletakkan sebagai variabel global yang dapat diakses oleh bagian program mana pun tanpa batasan. Contoh masalah fatal yang dapat terjadi adalah manipulasi saldo secara ilegal; pihak luar atau fungsi yang tidak sah dapat langsung mengubah nilai variabel saldo tanpa harus melewati proses verifikasi PIN yang benar. Dengan menerapkan OOP, risiko kebocoran dan manipulasi data tersebut dapat diminimalisir karena setiap interaksi terhadap data harus mematuhi aturan bisnis yang telah ditetapkan, sehingga integritas transaksi keuangan mahasiswa tetap terjaga.

2. Buatlah sebuah repositori GitHub publik. Lakukan inisialisasi Git lokal (*git init*), buat branch baru bernama *fitur-utama*, dan berikan screenshot riwayat (*log*) commit Anda yang rapi (minimal 3 commit) selama mengerjakan proyek ujian ini.

Jawab :

Proses manajemen proyek dilakukan dengan menggunakan Git sebagai sistem kendali versi lokal dan GitHub sebagai repositori publik. Langkah-langkah yang dilakukan adalah sebagai berikut:

- 1) Inisialisasi Repositori dan Branching

Melakukan inisialisasi folder proyek menjadi repositori Git lokal dan membuat branch baru sesuai instruksi dengan nama fitur-utama.

```
git init  
git checkout -b fitur-utama
```

2) Identitas Author

Mengatur konfigurasi identitas pengguna agar setiap perubahan tercatat

```
git config --global user.name "Rizky Augusta Gozzal"  
git config user.email "04231078@student.itk.ac.id"
```

3) Membuat File Laporan

```
Laporan_UTS_PBO.docx
```

4) Manajemen Commit

Melakukan penambahan file (*staging*) dan perekaman perubahan (*commit*) secara bertahap untuk memastikan riwayat pengerjaan proyek terdokumentasi dengan baik. Berikut adalah contoh perintah yang digunakan untuk mencapai minimal 3 *commit*:

```
# Commit 1  
git add .  
git commit -m "docs: inisialisasi file readme dan identitas proyek"  
  
# Commit 2  
git add .  
git commit -m "docs: analisis keamanan paradigma OOP vs Prosedural"  
  
# Commit 3  
git add .  
git commit -m "feat: merancang entitas Mahasiswa, Menu, dan StandMakanan"
```

5) Menghubungkan ke Repositori Publik (Push)

Menghubungkan repositori lokal ke GitHub dan mengirimkan seluruh perubahan di branch fitur-utama ke server.

git	remote	add	origin
https://github.com/augustagozzal/KANTIN-PAY_UTS-PBO.git			

6) Bukti Riwayat Commit (Log)

Berikut adalah tampilan riwayat commit yang telah dilakukan selama pengerjaan proyek:

```
Frisca Aulia@Morfine MINGW64 ~/KANTIN PAY ITK (fitur-utama)
$ git log --oneline
6bbbbfb (HEAD -> fitur-utama) feat: merancang entitas mahasiswa, menu, dan standmaka
nan
4f5efb4 docs: analisis keamanan paradigma OOP vs prosedural
789bffb docs inisialisasi file readme dan identitas proyek
```

TAHAP 2: PEMODELAN SISTEM

1. Rancangan Use Case Diagram

Use Case ini menggambarkan bagaimana Mahasiswa dan Stand Makanan berinteraksi dengan sistem KantinPay.

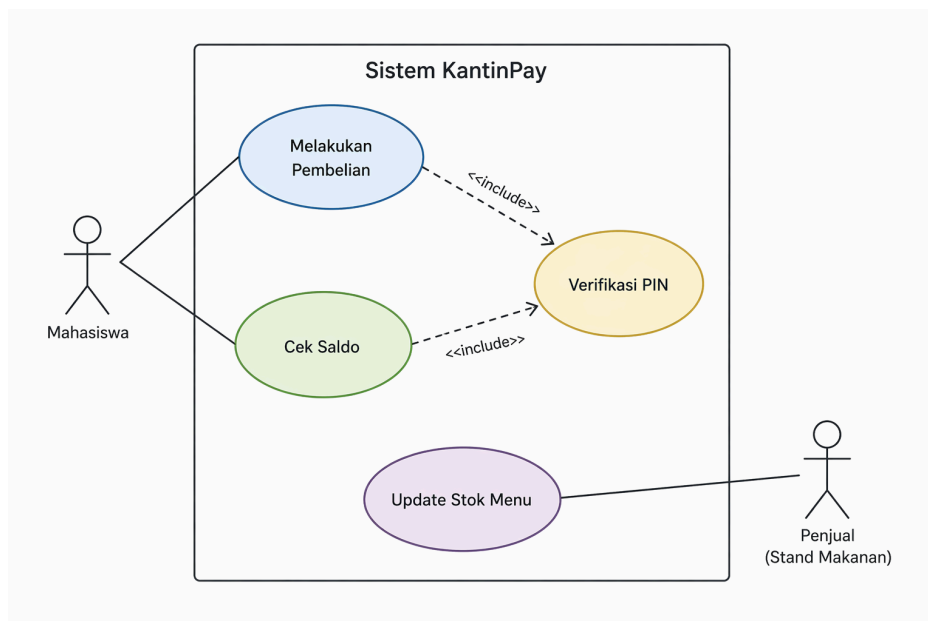
- Aktor Utama: Mahasiswa.
- Aktor Pendukung: Penjual (Stand Makanan).
- Daftar Use Case:

1. Melakukan Pembelian: (Include: Verifikasi PIN).

Penjelasan: Setiap kali Mahasiswa membeli, sistem wajib meminta PIN untuk keamanan.

2. Cek Saldo: (Include: Verifikasi PIN).

3. Update Stok Menu: (Dilakukan oleh Penjual).



2. Rancangan Class Diagram

Berdasarkan rancangan sistem KantinPay ITK, berikut adalah rincian class yang digunakan beserta penerapan prinsip Enkapsulasi:

A. Class Mahasiswa

Class ini berfungsi untuk mengelola data pengguna.

- Atribut:
 - nama (String): Nama lengkap mahasiswa.
 - pin (String): Kode keamanan akses (Modifier: Private).
 - saldo (Double): Saldo e-wallet mahasiswa (Modifier: Private).
- Tujuan Data Hiding: Atribut pin dan saldo disembunyikan agar tidak bisa diubah secara ilegal dari luar class. Perubahan saldo hanya bisa dilakukan melalui metode internal setelah validasi berhasil.

B. Class Menu

Class ini merepresentasikan item makanan atau minuman yang tersedia di kantin.

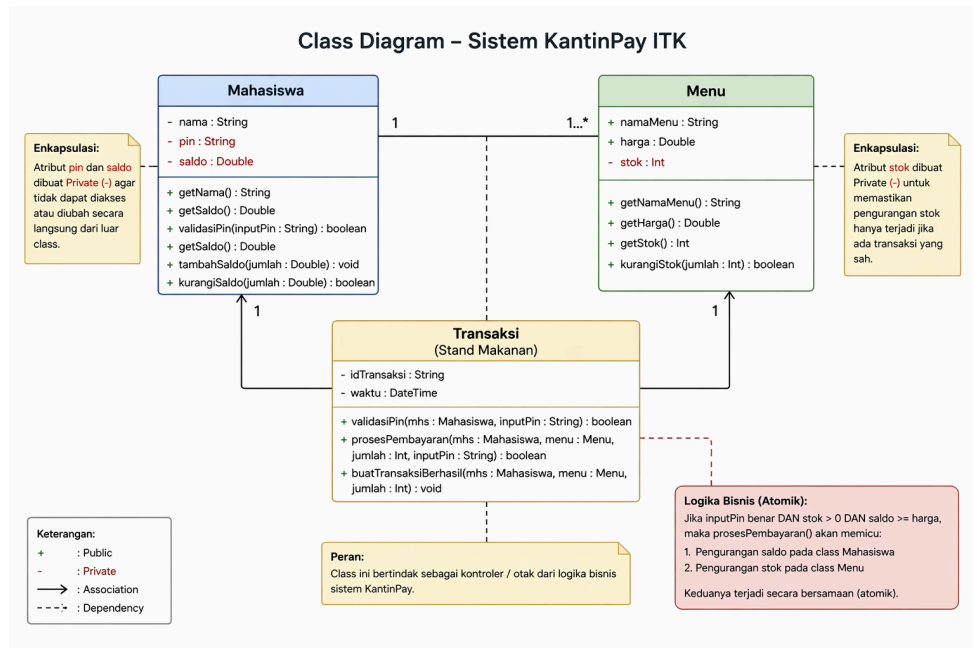
- Atribut:
 - namaMenu (String): Nama produk.
 - harga (Double): Harga per unit.
 - stok (Int): Jumlah ketersediaan barang (Modifier: Private).
- Tujuan Data Hiding: Atribut stok dibuat Private untuk memastikan bahwa pengurangan stok hanya terjadi jika ada transaksi yang sah, mencegah ketidakkonsistenan data antara sistem dan fisik.

C. Class Transaksi (Stand Makanan)

Class ini bertindak sebagai kontroler atau otak dari logika bisnis sistem KantinPay.

- Metode Utama:
 - validasiPin(): Mencocokkan input user dengan data di class Mahasiswa.
 - prosesPembayaran(): Logika untuk mengecek kecukupan saldo dan ketersediaan stok.

• Logika Bisnis: Jika inputPin benar DAN stok > 0 DAN saldo >= harga, maka metode ini akan memicu pengurangan saldo pada class Mahasiswa dan stok pada class Menu secara bersamaan (Atomik).



TAHAP 3: IMPLEMENTASI ENKAPSULASI KOTLIN

Pada tahap ini, rancangan sistem KantinPay ITK diimplementasikan ke dalam bahasa pemrograman Kotlin dengan menekankan pada aspek keamanan data (*Data Hiding*) dan aturan bisnis (*Business Rules*).

3.1 Kode Program (Main.kt)

Berikut adalah kode lengkap yang telah mengimplementasikan konsep Enkapsulasi dan Custom Setter/Validation:

```
/**
 * UTS PBO - Sistem KantinPay ITK
 * Nama : Rizky Augusta Gozzal
 * NIM : 04231078
 */

// 1. Implementasi Data Hiding pada Class Mahasiswa
class Mahasiswa(val nama: String, private val pin: String, saldoAwal: Double) {
    // Atribut saldo menggunakan 'private set' agar tidak bisa diubah sembarangan
    // dari luar class
    var saldo: Double = saldoAwal
        private set

    // Method untuk memvalidasi PIN secara internal
    fun cekPin(inputPin: String): Boolean {
        return inputPin == pin
    }

    // Method jalur resmi untuk memodifikasi saldo
    fun kurangiSaldo(jumlah: Double) {
        if (jumlah <= saldo) {
            saldo -= jumlah
        }
    }
}
```



```

    }
}

// 2. Implementasi Data Hiding pada Class Menu
class Menu(val namaProduk: String, val harga: Double, stokAwal: Int) {
    // Atribut stok tidak boleh dimodifikasi langsung dengan operator '=' di luar
class
    var stok: Int = stokAwal
    private set

    // Method jalur resmi untuk update stok
    fun updateStok() {
        if (stok > 0) {
            stok--
        }
    }
}

// 3. Class StandMakanan untuk Menangani Validasi Aturan Bisnis (Custom
Setter)
class StandMakanan(val namaStand: String) {

    fun prosesTransaksi(pembeli: Mahasiswa, produk: Menu, inputPin: String) {
        println("--- Transaksi di $namaStand ---")
        println("Mencoba membeli  ${produk.namaProduk}  seharga
Rp${produk.harga}")

        // Validasi 1: Keamanan (PIN)
        if (!pembeli.cekPin(inputPin)) {
            println("[HASIL] GAGAL: PIN yang dimasukkan salah!")
            println("-----\n")
            return
        }
    }
}

```

```

    }

    // Validasi 2: Ketersediaan (Stok)
    if (produk.stok <= 0) {
        println("[HASIL] GAGAL: Stok ${produk.namaProduk} sudah habis!")
        println("-----\n")
        return
    }

    // Validasi 3: Kelayakan (Saldo)
    if (pembeli.saldo < produk.harga) {
        println("[HASIL] GAGAL: Saldo tidak cukup (Saldo saat ini:
Rp${pembeli.saldo})")
        println("-----\n")
        return
    }

    // Eksekusi jika semua validasi lolos (Simulasi SUKSES)
    pembeli.kurangiSaldo(produk.harga)
    produk.updateStok()
    println("[HASIL] SUKSES: Pembayaran Berhasil!")
    println("Update Saldo: Rp${pembeli.saldo} | Update Stok: ${produk.stok}")
    println("-----\n")
}
}

// 4. Simulasi di Main Function
fun main() {
    // Inisialisasi Objek (Sesuai Aturan Bisnis)
    val userRizky = Mahasiswa("Rizky Augusta", "04231078", 20000.0)
    val menuAyam = Menu("Ayam Geprek", 15000.0, 1) // Stok disetel 1 untuk
pengujian
    val kantin = StandMakanan("Kantin Pusat ITK")

```

```
// Skenario 1: GAGAL (Pelanggaran Keamanan - PIN Salah)
kantin.prosesTransaksi(userRizky, menuAyam, "123456")

// Skenario 2: SUKSES (Transaksi Sah)
kantin.prosesTransaksi(userRizky, menuAyam, "04231078")

// Skenario 3: GAGAL (Pelanggaran Stok - Stok Habis)
kantin.prosesTransaksi(userRizky, menuAyam, "04231078")

// Skenario 4: GAGAL (Pelanggaran Saldo - Saldo Kurang)
val menuLain = Menu("Es Teh", 7000.0, 10)
kantin.prosesTransaksi(userRizky, menuLain, "04231078")
}
```

3.2 Analisis Hasil Eksekusi

Berdasarkan simulasi yang dilakukan pada fun main(), sistem telah berhasil memenuhi kriteria berikut:

1. Data Hiding Berhasil: Variabel sensitif seperti saldo dan stok menggunakan private set. Ini terbukti karena modifikasi data hanya bisa dilakukan melalui fungsi prosesTransaksi() dan tidak bisa diubah langsung menggunakan operator =.
2. Skenario Gagal Terpenuhi:
 - Sistem menolak transaksi saat PIN salah (Keamanan).
 - Sistem menolak transaksi saat stok item bernilai 0 (Logika Stok).
 - Sistem menolak transaksi saat saldo pembeli di bawah harga menu (Logika Saldo).
3. Skenario Sukses Terpenuhi: Saat semua data valid, saldo dan stok berkurang secara sinkron dan otomatis.

Gambar hasil simulasi :

```
--- Transaksi di Kantin ITK ---  
[HASIL] GAGAL: PIN Salah!  
  
--- Transaksi di Kantin ITK ---  
[HASIL] SUKSES! Sisa Saldo: 5000.0  
  
--- Transaksi di Kantin ITK ---  
[HASIL] GAGAL: Stok Habis!
```